End to End 3D Navigation Framework (Nav3D) for Quadruped Robots via Reward Shaping and Curriculum Design

***Abstract*—Navigating complex 3D environments is a critical capability for quadruped robots, enabling them to perform tasks in real-world settings with varied terrain and obstacles. However, training such robots with deep reinforcement learning is challenging due to sparse rewards and high-dimensional control spaces. This paper presents a reinforcement learning framework to address these challenges by training a quadruped robot to perform 3D navigation tasks. We adopt a curriculum learning strategy that gradually increases task difficulty, allowing the policy to progressively master navigation toward distant and complex targets. Additionally, we design a reward function inspired by the concept of a progress estimator to provide continuous, informative feedback related to the navigation goal. Ablation studies show that both the learning curriculum and progress estimator reward are crucial for training stability and task success. The resulting policy demonstrates robust performance across 3D navigation scenarios, effectively controlling both position and height.**

***Index Terms*—Reinforcement Learning, Whole-Body Motion Planning and Control, Machine Learning for Robot Control, Legged Robots, Robot Learning.**

I. Introduction

Legged robots are gaining attention for their adaptability and ability to operate in complex, irregular environments. High degrees of freedom (DOF) and flexibility make them suitable for tasks in military, disaster response, geological exploration, and other challenging domains [1]–[3]. Among them, quadruped robots offer a balance between simplicity and stability—simpler design than hexapods and more stable than bipeds [2].

Recent studies have advanced quadruped locomotion using model-based approaches such as Model Predictive Control (MPC) for dynamic gait control on Cheetah robot [4], or confined spaces navigation framework [5], [6]. More recently, deep reinforcement learning (DRL) rapidly progresses locomotion tasks, including uneven terrain [7]–[9], confined environments [10], non-rigid surfaces [11], and jumping [12], while also enabling robust transfer learning [13]–[15]. DRL eliminates the need for complex dynamic models by learning optimal behaviors through interaction with the environment. It can handle high-dimensional input and output spaces and adapt to unknown scenarios without manual tuning, offering a scalable solution for complex tasks.

DRL has also enabled complex navigation behaviors, such as goal-directed movement with parkour-like actions [16], and operation in challenging environments with wheeled-legged robots [17]. Hierarchical framework integration to DRL splits control into high-level decision-making and low-level locomotion. While this decomposition helps manage complexity, it introduces limitations such as failures due to incorrect high-level decisions and prolonged training times [16]. Moreover, the separation may constrain the solution space [18]. End-to-end policies can unify navigation and locomotion within a single network, avoiding explicit task decomposition and reducing error propagation between layers [18], [19].

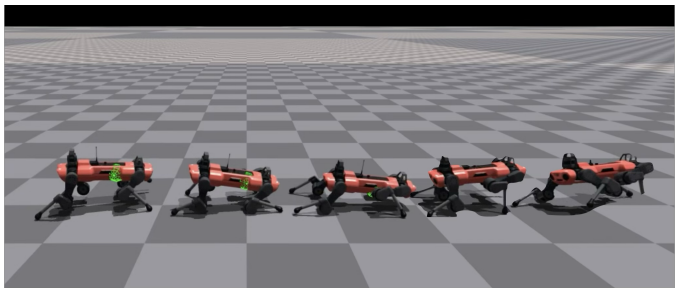


Fig. 1. Legged robot 3D navigation represented as a sequence of waypoints. The green spheres indicate the goal position, comprising both planar coordinates and torso height.

In navigation tasks, sparse rewards pose a key challenge. This sparsity can lead to suboptimal behavior policy stuck in a local optimum [20]. This problem becomes more pronounced in navigation with end-to-end policies where a single policy handles both navigation and locomotion, making the reward space (goal position) distant from the action space (joint positions). This results in inefficient training, often requiring a large number of steps to achieve convergence [19].

Several strategies have been proposed to address reward sparsity in end-to-end policy learning for quadruped robot navigation. Reward shaping adds an exploration reward to guide the agent toward the goal. While this approach helps mitigate the sparse-reward issue by encouraging the agent to explore relevant states, it still suffers from training instability [18]. Curriculum learning tackles the problem by first training the agent on simpler tasks and gradually increasing task complexity [19].

While reward shaping and curriculum learning have been proposed to address these issues, previous works focus on 2D navigation tasks in challenging environments and do not evaluate how these strategies affect training stability, and learning efficiency—particularly in a higher-dimensional navigation task. This leaves a critical gap in understanding how to scale end-to-end navigation to 3D goal spaces that require both planar and vertical alignment.

In this paper, we address this gap by extending end-to-end policy learning for quadruped navigation to a 3D goal space, where the agent must align both its planar (x–y) position and torso height with the target. To overcome the sparse reward challenge, we incorporate a progress estimator inspired by [21] directly into the reward function, enabling more consistent feedback during training. In addition, we design a curriculum

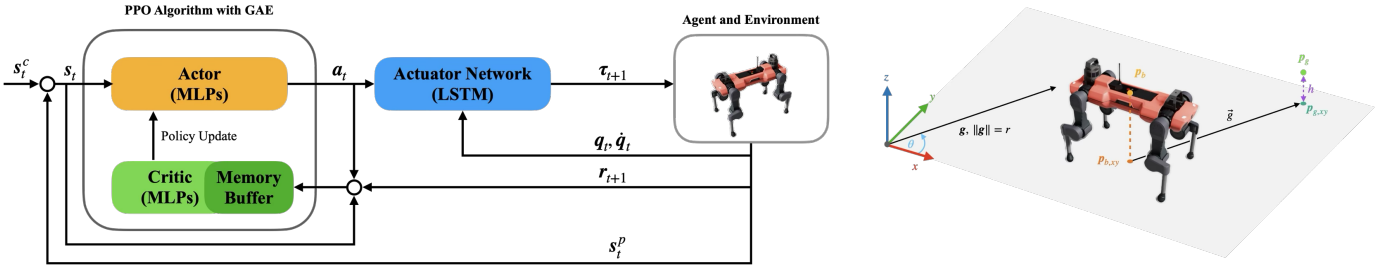


Fig. 2. **Overview of the Framework** (left): At each time step t, the observation $\boldsymbol{s}_t$ consists of proprioceptive measurements ($\boldsymbol{s}_t^p$) and input commands ($\boldsymbol{s}_t^c$), represented as $\boldsymbol{s}_t = [\boldsymbol{s}_t^p, \boldsymbol{s}_t^c]$. The actor policy network selects an action $\boldsymbol{a}_t$, corresponding to the desired joint positions. A pre-trained actuator network [8] receives the selected action along with the current joint positions $\boldsymbol{q}_t$ and joint velocities $\dot{\boldsymbol{q}}_t$. It acts as a low-level controller to compute the motor torques $\boldsymbol{\tau}_{t+1}$. The entire framework operates at 50 Hz to select actions and update motor torques. **Frame Representation for Generating the Target Position** (right): The coordinate frame shown is the environment frame. The robot's position is denoted by $\boldsymbol{p}_g$. To generate the target position, we first project the robot's position onto the XY plane. We then add the vector $\boldsymbol{g}$ to this projection. Finally, we raise the target position by a height of h meters above the ground.

learning strategy tailored for navigation tasks involving higher-dimensional goals.

This study presents a novel 3D navigation framework for quadruped robots, requiring the agent to perform both planar movement and dynamic torso height adjustment in response to input commands. We conduct a comprehensive set of ablation experiments in simulation to analyze the effects of reward shaping and curriculum design on training stability, convergence speed, and the resulting policy behaviors. Finally, we validate the trained policy in simulated 3D navigation scenarios.

II. Method

A. State and Action Space

The state space ($\boldsymbol{s}$) is defined as a combination of the robot's proprioceptive measurements ($\boldsymbol{s_p}$) and input commands ($\boldsymbol{s_c}$), as shown in Table I. The proprioceptive measurements include base linear velocity, base angular velocity, joint positions, joint velocities, and the previous action. The command input specifies a 3D target position, denoted as $\boldsymbol{c} = \boldsymbol{p}_g = [p_{g,x}, p_{g,y}, p_{g,z}] \in \mathbb{R}^3$. This goal position is computed by sampling three parameters randomly: radius (r), which refers to the distance from the robot to the goal position; angle (θ), which defines the direction of the target relative to the environment frame; and height (h), which specifies the vertical offset from the ground. The goal position in the environment frame is then calculated as

$$\boldsymbol{p}_g = \begin{bmatrix} p_{g,x} \\ p_{g,y} \\ p_{g,z} \end{bmatrix} = \begin{bmatrix} p_{b,x} \\ p_{b,y} \\ 0 \end{bmatrix} + \begin{bmatrix} r\cos(\theta) \\ r\sin(\theta) \\ h \end{bmatrix}, \tag{1}$$

where $p_{b,x}$ and $p_{b,y}$ are the current position of the robot on x and y axes. No artificial noise is added to the observations.

The action space, $\boldsymbol{a} \in \mathbb{R}^{12}$, is defined as the desired joint positions for all 12 joints of the quadruped robot. A pre-trained actuator network, as introduced in [8], is used to compute the motor torques. This network takes the selected action along with the measured joint positions and velocities as input, and outputs the control torques $\boldsymbol{\tau} \in \mathbb{R}^{12}$.

TABLE I
Components of the Observation Space, their Dimensions, and Associated Symbols.

Type	Element	Dimension	Symbol
Command	Target Position (m)	3	$\boldsymbol{p}_g$
Proprioception	Base Linear Velocity (m/s)	3	$\boldsymbol{v}_b$
	Base Angular Velocity (rad/s)	3	$\boldsymbol{\omega}_b$
	Measured Joint Positions (rad)	12	$\boldsymbol{q}$
	Measured Joint Velocities (rad/s)	12	$\dot{\boldsymbol{q}}$
	Previous Action (rad)	12	$\boldsymbol{a}'$

B. Rewards

We design the reward function for our 3D navigation task to encourage the agent to learn both navigation and locomotion skills necessary to reach the target effectively. The reward is divided into two groups: the *Navigation Task Group*, which drives the robot to reach the target position, and the *Locomotion Task Group*, which enforces natural and efficient movement behavior. To mitigate sparse reward signals, we incorporate a progress estimator inspired by [21], which shapes the position and height tracking rewards. This estimator provides continuous feedback based on incremental improvement toward the goal, complementing the sparse nature of the main task reward—reaching the goal position. The complete set of reward terms and their corresponding weights is summarized in Table II.

1) Navigation Task Group:

a) Reach Goal Reward: A sparse reward triggered when the robot reaches the target within a predefined acceptance range. Due to its sparse nature, it is received only at the time step immediately before the target is resampled during the episode. This term is defined as

$$r_{reach} = \mathbb{1}_{\|\boldsymbol{p}_g - \boldsymbol{p}_b\| \leq \delta} \tag{2}$$

where $\boldsymbol{p}_g$ and $\boldsymbol{p}_b$ denote the positions of the goal and the robot's base in the world coordinate system, respectively. The scalar δ represents the allowed positional error, set to 0.1 meters.

b) Position Tracking Reward: To address the sparse nature of the goal-reaching reward, we introduce a shaping term using the aforementioned progress estimator, which rewards

progress toward the target position in the x-y plane. This term reflects how close the robot is to the target and is defined as

$$r_{position} = e^{\frac{-\|\boldsymbol{p}_{g,xy} - \boldsymbol{p}_{b,xy}\|^2}{\beta}} \tag{3}$$

where $\boldsymbol{p}_{g,xy}$ and $\boldsymbol{p}_{b,xy}$ are the x-y components of the target and robot base positions, respectively, in the world frame. The hyperparameter β balances this reward against other terms and is set to 1.5 in our work.

c) Height Tracking Reward: Similar to the position tracking reward, this term encourages the robot to maintain its base height near the target's vertical position. We treat height separately under the assumption that vertical control differs significantly from planar motion. This reward is described as

$$r_{height} = e^{\frac{-\|p_{g,z} - p_{b,z}\|^2}{\phi}} \tag{4}$$

where $p_{g,z}$ and $p_{b,z}$ are the z-axis (height) components of the goal and the robot's base position in the world frame, respectively. The hyperparameter ϕ is set to 0.01.

d) Guiding Reward: Inspired by Rudin et al. [8], the guiding reward term encourages movement in the direction of the goal. It is defined as

$$r_{guiding} = \frac{\boldsymbol{v}_b \cdot (\boldsymbol{p}_g - \boldsymbol{p}_b)}{\|\boldsymbol{v}_b\| \, \|\boldsymbol{p}_g - \boldsymbol{p}_b\|} \tag{5}$$

where $\boldsymbol{v}_b$ is the linear velocity of the robot's base in the world frame.

e) Stalling Penalty: This term penalizes the robot for remaining relatively stationary when the goal has not yet been reached. The penalty is calculated as

$$r_{stall} = \mathbb{1}_{\|\boldsymbol{v}_{b,xy}\| < \psi} \cdot \mathbb{1}_{\|\boldsymbol{p}_{g,xy} - \boldsymbol{p}_{b,xy}\| > \varphi} \tag{6}$$

where $\boldsymbol{v}_{b,xy}$ is the linear velocity of the robot's base in the x–y plane in the world frame. The hyperparameters ψ and φ define the conditions that trigger the penalty: the robot is moving slower than 0.1 m/s and is still more than 0.1 m away from the goal.

2) Locomotion Task Group: The locomotion task group encourages efficient, stable, and physically realistic behavior. Following the design in [8], it consists of penalty terms applied to vertical linear velocity, roll and pitch angular velocities, joint torque, action rate, and number of collisions. Additionally, it includes a positive reward term for foot air-time, which promotes more dynamic and desirable locomotion behavior.

C. Episode Process

Each episode starts by spawning the agent in the Each episode begins by placing the agent in the simulation environment with a randomly generated target position as the initial navigation waypoint. The variables for calculating the target position are randomly sampled within the range defined by the current curriculum level. Details on the curriculum learning setup are provided in Section II-D. The episode can terminate under two conditions: 1) if the robot falls and its base makes contact with the ground, or 2) if the episode's time limit is reached. However, if the robot successfully reaches the target position before the timeout, the environment re-samples a new target position for the subsequent stage within the same episode, allowing for continuous movement.

TABLE II
SUMMARY OF REWARD TERMS USED DURING TRAINING, CATEGORIZED BY NAVIGATION AND LOCOMOTION OBJECTIVES.

Group	Reward Term	Expression	Weight
Navigation	Reach Goal Reward	$\mathbb{1}_{\|\boldsymbol{p}_g - \boldsymbol{p}_b\| \le \delta}$	$300dt$
	Position Tracking Reward	$e^{\frac{-\|\boldsymbol{p}_{g,xy} - \boldsymbol{p}_{b,xy}\|^2}{1.5}}$	$1dt$
	Height Tracking Reward	$e^{\frac{-\|p_{g,z} - p_{b,z}\|^2}{0.01}}$	$1dt$
	Guiding Reward	$\frac{\boldsymbol{v}_b \cdot (\boldsymbol{p}_g - \boldsymbol{p}_b)}{\|\boldsymbol{v}_b\| \, \|\boldsymbol{p}_g - \boldsymbol{p}_b\|}$	$1dt$
	Stalling Penalty	$\mathbb{1}_{\|\boldsymbol{v}_{b,xy}\| < \psi} \cdot \mathbb{1}_{\|\boldsymbol{p}_{g,xy} - \boldsymbol{p}_{b,xy}\| > \varphi}$	$-1dt$
Locomotion	Vertical Velocity Penalty	$v_{b,z}^2$	$-4dt$
	Angular Velocity Penalty	$\|\boldsymbol{\omega}_{b,xy}\|^2$	$-0.05dt$
	Torque Penalty	$\|\boldsymbol{\tau}\|^2$	$-0.00002dt$
	Action Rate Penalty	$\|\boldsymbol{a} - \boldsymbol{a}'\|^2$	$-0.25dt$
	Collision Penalty	$n_{\text{collision}}$	$-0.001dt$
	Feet Air-Time Reward	$\sum_{f=1}^{4}(t_{\text{air},f} - 0.5)$	$1dt$

TABLE III
PPO HYPERPARAMETERS AND POLICY NETWORK ARCHITECTURE

Parameter	Value
Batch Size	$196,608 (4096 \times 48)$
Mini-batch Size	$49,152 (4096 \times 12)$
Epochs	5
Clip Range	0.2
Discount Factor	0.99
GAE Discount Factor	0.95
Learning Rate	adaptive
delta time-step (dt)	0.02 s
Neurons per Hidden Layers	$(512, 256, 128)$
Activation Function	ELU

D. Curriculum Learning

To address the challenges of sparse rewards and improve training success, we adopt a curriculum learning strategy inspired by [22]. We implement a learning curriculum by gradually increasing the difficulty of the navigation task. The curriculum adjusts based on the agent's success rate, defined as reaching at least one target within an episode. When the success rate exceeds 80 percent, the curriculum level increases by increasing the target distance range by 0.25 m and slightly reducing the target height range by 0.01 m for the randomly sampled command variables. The radius and height are constrained within $[1.0, 6.0]$ m and $[0.2, 0.4]$ m, respectively. This results in a total of 20 curriculum levels.

E. Training Details

We train the policy using Proximal Policy Optimization (PPO) with Generalized Advantage Estimation (GAE) [23], using the hyperparameters listed in Table III. The simulation is built on Isaac Gym [24], and training is conducted using the parallel setup described in [8], for a total of 1000 iterations. The quadruped robot model used is ANYmal-C. Both actor and critic networks are implemented as multi-layer perceptrons (MLPs); architectural details are listed in Table III. To enhance sample efficiency and reduce training instability, we adopt short episodes (6 s) and a large batch size of 196,608, following the guidelines suggested in [8].

III. RESULT

This section presents our experimental setup and evaluation of the proposed method. We first conduct ablation studies to examine the impact of individual training components on

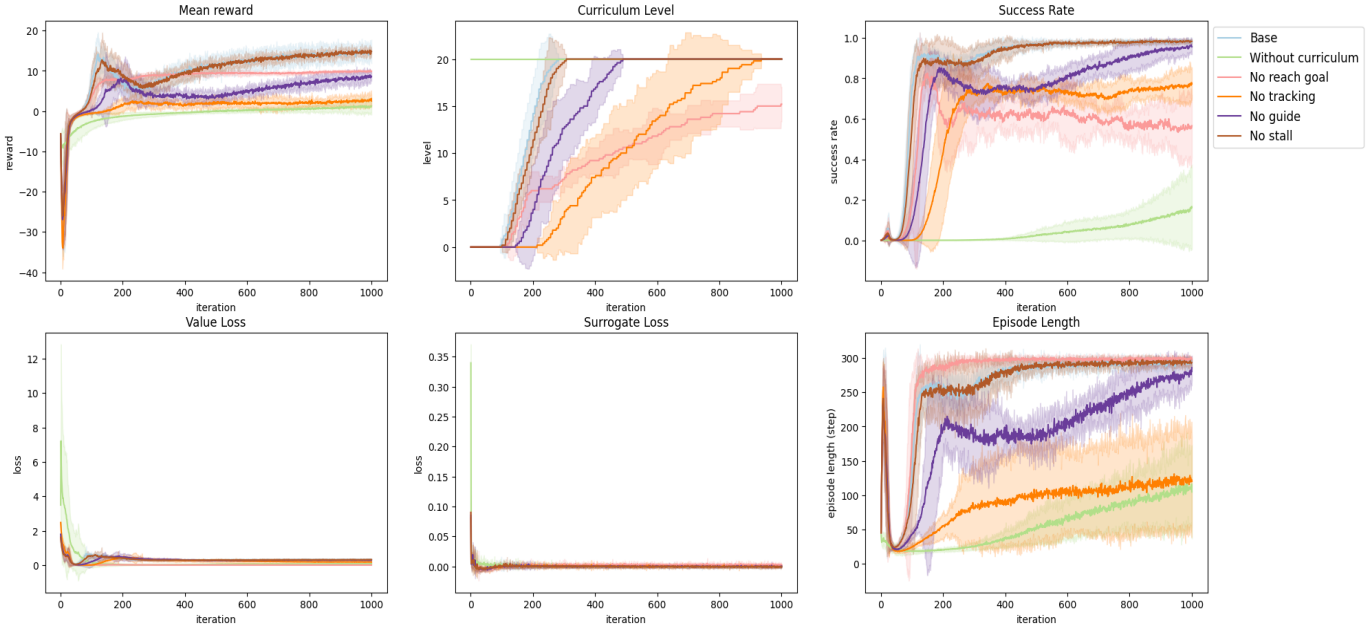


Fig. 3. The ablation results across six metrics: mean reward, value loss, surrogate loss, curriculum level, success rate, and episode length. Each configuration was trained with five random seeds; the solid line indicates the mean and the shaded area represents the 95 percent confidence interval.

policy performance in Section III-A. We then evaluate the learned policy in 3D navigation tasks for four scenarios which detail in Section III-B to validate its effectiveness across varying spatial conditions.

A. Ablation Experiments

To understand the contribution of individual training factors to the overall policy performance, we perform an ablation experiment to investigate the effect of each reward term in the navigation group and learning curriculum on training stability, convergence speed and policy behavior. Training follows the procedure described in Section II, using six different setups described below. All policies are trained using five random seeds to account for variability in the learning process.

Base: The base setup is a baseline which activates all reward terms and uses the learning curriculum for training.

Without curriculum: In this setup, policies are trained with a fully random range of distances and heights of the target position from the start throughout training.

No reach goal, tracking, guide, and stall: We separately deactivate each reward term, including reach goal, tracking group (position and height), guide, and stall rewards.

In Figure 3, we compare six different training setups over the training of 1000 iterations. We measure mean reward, value loss, surrogate loss, curriculum level, success rate, and episode length to investigate the impact of each training factor on training efficiency and 3D navigation performance.

1) Curriculum Learning: In this setting, the agent is exposed to randomly sampled target positions across the full range of distances and heights from the beginning of training. This introduces high task complexity early on, making it difficult for the agent to explore and learn effectively.

As shown in Figure 3, removing curriculum learning significantly affects the high-value loss and leads to greater variance across different random seeds. Without intermediate goals or a structured progression, the agent rarely encounters states that provide meaningful feedback. As a result, the critic network—which estimates the expected future rewards—struggles to learn effectively. When rewards are sparse, its predictions become inaccurate and differ from the actual rewards received.

The absence of curriculum leads to poor navigation behavior. Agents frequently fail to reach the target, resulting in lower success rates and shorter episode lengths. A common failure pattern observed is the agent ”leaning” excessively toward the target, which often causes it to fall before making significant progress, , as illustrated in Figure 4. This behavior highlights the difficulty of learning long-range goal-reaching tasks under sparse rewards without structured guidance.

Overall, these results underscore the importance of curriculum learning for 3D navigation tasks. By gradually increasing the difficulty of the task—starting with closer and more attainable goals—the curriculum allows the agent to first learn basic locomotion and navigation strategies before tackling more complex scenarios. This suggests that the absence of curriculum learning leads to instability in the training process, slow convergence, and suboptimal behavior.

2) Reach Goal Reward: Removing the reach goal reward greatly reduced success rates, as evidenced by the success rate block in Figure 3. As a result, after 1,000 training iterations, the agent was only able to progress to an average curriculum level with a distance of 4.8 meters—down from the maximum of 6 meters—and a base height of 0.248 m, compared to the minimum base height of 0.2 m. The agent developed atypical behavior: it would begin moving toward the target but then attempt to approach it gradually by tilting forward, rather than fully reaching the goal. This behavior appears to result from the agent’s focus on maximizing tracking rewards, which



Fig. 4. Motion snapshot illustrating the leaning behavior observed before reaching the target. The green sphere indicates the target position with the acceptable tolerance range.

provide positive feedback for staying near the target, rather than completing the task and triggering the next goal. These findings highlight the critical role of the reach goal reward in helping the agent learn what it means to successfully reach the target—not merely approach it closely.

3) Tracking Reward Group: Removing the tracking reward group leads to high variance in curriculum level, success rate, and episode length across different random seeds, indicating less stable training. The tracking reward helps reduce the problem of sparse rewards by giving the agent a continuous signal about its progress toward the goal. Without this reward, episodes often became very short because the agent would quickly reach the first target and then fall, as it did not get any reward related to moving toward the next target. Thus, the tracking reward group is essential for training stability and helps the agent keep moving toward future goals.

4) Guiding Reward: As shown in Figure 3, the guiding reward helps speed up learning. It allows the agent to reach the highest curriculum level and converge faster compared to the Base setup. However, after 1,000 training iterations, the overall performance was similar to the Base setup. Regarding the agent's behavior, the actions taken without the guiding reward were mostly the same as in the baseline policy.

5) Stall Reward: Removing the stall penalty did not result in a significant difference in training performance or policy behavior compared to the baseline configuration.

B. Navigation and Locomotion Skills

We evaluate the baseline policy's 3D navigation capability across four scenarios designed to assess spatial control under varying constraints. Each scenario is evaluated over 100 trials, with success defined as the robot not falling and reaching within a 0.1 m radius of the target position.

Scenario 1: Planar Navigation (XY-Only): Targets are sampled randomly in the horizontal plane at a fixed height of $h = 0.45$ m, with radius $r \in [1, 6]$ m to benchmarks planar locomotion.

Scenario 2: Vertical Movement (Z-Only): Targets are placed directly above or below the robot with height offset $h \in [0.2, 0.4]$ m, assessing the agent's vertical base motion capabilities.

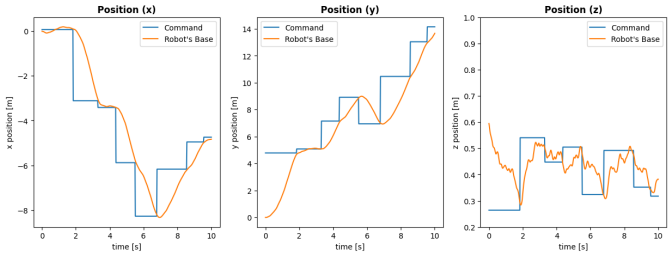


Fig. 5. Representative position tracking (relative to the environment frame) of the robot's base and command, conducted under the same configuration as Scenario 4 (Waypoint Navigation), but extended to a 10-second duration.

Scenario 3: Full 3D Navigation: Targets are sampled in full 3D space with radius $r \in [1, 6]$ m and height $h \in [0.2, 0.4]$ m, requiring coordination across all spatial dimensions.

Scenario 4: Waypoint Navigation: The robot is tasked with reaching a sequence of 3D waypoints within 6 s. Targets follow the same distribution as Scenario 3 and evaluate consistency in waypoint navigation.

Success rates for each scenario are summarized in Table IV. The policy demonstrates high robustness and generalization capability across all evaluated navigation tasks.

TABLE IV
SUCCESS RATE ACROSS NAVIGATION SCENARIOS

Scenario	Success Rate
1 (XY-Only)	0.99
2 (Z-Only)	1.00
3 (3D Navigation)	1.00
4 (Waypoint Navigation)	0.99

Fig. 5 illustrates a representative trajectory and command profile from Scenario 4. While the robot demonstrates successful 3D waypoint tracking and navigation, some limitations remain. The robot sometimes overshot the targets, especially when turning from the opposite direction. Small jerks were also noticed when the robot was stationary, suggesting some issues with steady-state control.

IV. CONCLUSION AND DISCUSSION

Our ablation results indicate that the learning curriculum, tracking reward group, and reach goal reward are essential components for successful training in 3D navigation tasks. Among them, the curriculum plays a critical role in addressing the sparse reward challenge, enabling the policy to learn progressively from easier to harder goals. In terms of reward shaping, the tracking reward group is especially important for training stability. Without the reach goal reward, the agent lacks a clear signal to finalize its motion. This confirms that a terminal reward is necessary to anchor the learning objective. The tracking reward group facilitates continuous motion during waypoint navigation. The guiding reward was found to accelerate early training, though its contribution to final performance was minimal. The stalling penalty had little effect, suggesting it may not be critical in this context.

Despite the framework's ability to generalize across 3D goal spaces, it exhibits limitations in handling fine-grained

control near the goal. Jerky movements and overshooting indicate room for improvement in the final positioning behavior. Additionally, the policy struggles when presented with targets outside the training distribution, highlighting a need for better generalization.

Future work will explore strategies to reduce jerky terminal behavior and improve steady-state control. We also aim to extend the policy's generalization capability to handle out-of-distribution goals and investigate sim-to-real transfer for deployment on physical quadruped platforms.

References

[1] X. Meng, S. Wang, Z. Cao, and L. Zhang, "A review of quadruped robots and environment perception," in *2016 35th Chinese Control Conference (CCC)*, IEEE, 2016, pp. 6350–6356.

[2] J. He, J. Shao, G. Sun, and X. Shao, "Survey of quadruped robots coping strategies in complex situations," *Electronics*, vol. 8, no. 12, p. 1414, 2019.

[3] Z. Wang and H. Gu, "A review of locomotion mechanisms of urban search and rescue robot," *Industrial Robot: An International Journal*, vol. 34, no. 5, pp. 400–411, 2007.

[4] G. Bledt, M. J. Powell, B. Katz, J. Di Carlo, P. M. Wensing, and S. Kim, "Mit cheetah 3: Design and control of a robust, dynamic quadruped robot," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2018, pp. 2245–2252.

[5] Y. Chen, L. Lian, Y.-H. Hsieh, Q. Nguyen, and S. K. Gupta, "Informed sampling-based planning to enable legged robots to safely negotiate permeable obstacles," *Journal of Mechanisms and Robotics*, vol. 15, no. 5, p. 051 002, 2023.

[6] R. Buchanan, L. Wellhausen, M. Bjelonic, T. Bandyopadhyay, N. Kottege, and M. Hutter, "Perceptive whole-body planning for multilegged robots in confined spaces," *Journal of Field Robotics*, vol. 38, no. 1, pp. 68–84, 2021.

[7] X. B. Peng, G. Berseth, and M. Van de Panne, "Terrain-adaptive locomotion skills using deep reinforcement learning," *ACM Transactions on Graphics (TOG)*, vol. 35, no. 4, pp. 1–12, 2016.

[8] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Conference on Robot Learning*, PMLR, 2022, pp. 91–100.

[9] G. Bellegarda, Y. Chen, Z. Liu, and Q. Nguyen, "Robust high-speed running for quadruped robots via deep reinforcement learning," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2022, pp. 10 364–10 370.

[10] T. Miki, J. Lee, L. Wellhausen, and M. Hutter, "Learning to walk in confined spaces using 3d representation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2024, pp. 8649–8656.

[11] T. Kim and S.-H. Lee, "Quadruped locomotion on non-rigid terrain using reinforcement learning," *arXiv preprint arXiv:2107.02955*, 2021.

[12] G. Bellegarda, C. Nguyen, and Q. Nguyen, "Robust quadruped jumping via deep reinforcement learning," *Robotics and Autonomous Systems*, vol. 182, p. 104 799, 2024.

[13] L. Smith, I. Kostrikov, and S. Levine, "A walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning," *arXiv preprint arXiv:2208.07860*, 2022.

[14] T. Haarnoja, S. Ha, A. Zhou, J. Tan, G. Tucker, and S. Levine, "Learning to walk via deep reinforcement learning," *arXiv preprint arXiv:1812.11103*, 2018.

[15] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Science robotics*, vol. 7, no. 62, eabk2822, 2022.

[16] D. Hoeller, N. Rudin, D. Sako, and M. Hutter, "Anymal parkour: Learning agile navigation for quadrupedal robots," *Science Robotics*, vol. 9, no. 88, eadi7566, 2024.

[17] J. Lee, M. Bjelonic, A. Reske, L. Wellhausen, T. Miki, and M. Hutter, "Learning robust autonomous navigation and locomotion for wheeled-legged robots," *Science Robotics*, vol. 9, no. 89, eadi9641, 2024.

[18] N. Rudin, D. Hoeller, M. Bjelonic, and M. Hutter, "Advanced skills by learning locomotion and local navigation end-to-end," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2022, pp. 2497–2503.

[19] Z. Xu, A. H. Raj, X. Xiao, and P. Stone, "Dexterous legged locomotion in confined 3d spaces with reinforcement learning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2024, pp. 11 474–11 480.

[20] J. Hare, "Dealing with sparse rewards in reinforcement learning," *arXiv preprint arXiv:1910.09281*, 2019.

[21] L. Matignon, G. J. Laurent, and N. Le Fort-Piat, "Reward function and initial values: Better choices for accelerated goal-directed reinforcement learning," in *International Conference on Artificial Neural Networks*, Springer, 2006, pp. 840–849.

[22] E. Alonso, M. Peter, D. Goumard, and J. Romoff, "Deep reinforcement learning for navigation in aaa video games," *arXiv preprint arXiv:2011.04764*, 2020.

[23] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[24] V. Makoviychuk, L. Wawrzyniak, Y. Guo, *et al.*, "Isaac gym: High performance gpu-based physics simulation for robot learning," *arXiv preprint arXiv:2108.10470*, 2021.